Punch Out Model Synthesis: A Stochastic Algorithm for Constraint Based Tiling Generation

Zzyv Zzyzek [1]

Abstract

As an artistic aid in tiled level design, Constraint Based Tiling Generation (CBTG) algorithms can help to automatically create level realizations from a set of tiles and placement constraints. Merrell's *Modify in Blocks Model Synthesis* (MMS) and Gumin's *Wave Function Collapse* (WFC) have been proposed as Constraint Based Tiling Generation (CBTG) algorithms that work well for many scenarios but have limitations in problem size, problem setup and solution biasing. We present Punch Out Model Synthesis (POMS), a Constraint Based Tiling Generation algorithm, that can handle large problem sizes, requires minimal assumptions for setup and can help mitigate solution biasing. POMS attempts to resolve indeterminate grid regions by trying to progressively realize sub-blocks, performing a stochastic boundary erosion on previously resolved regions should sub-block resolution fail. We highlight the results of running a reference implementation on different tile sets and discuss a tile correlation length, implied by the tile constraints, and its role in choosing an appropriate block size to aid POMS in successfully finding grid realizations.

arXiv:2501.14786v1 [cs.DC] 5 Jan 2025

1. Introduction

1.1. Overview

We present Punch Out Model Synthesis (POMS), an algorithm that works on a regular 2D or 3D grid to find a tile placement realization subject to pairwise tile constraints in each grid direction ($\pm X, \pm Y, \pm Z$).

POMS is a grid level stochastic Constraint Based Tiling Generation (CBTG) algorithm whose primary benefits are:

- Requires minimal assumptions on initial setup state
- Has resources that scale primarily with block size and not grid size
- Can reliably find realizations on arbitrarily sized grids with tile constraints that have finite correlation length

and primary drawbacks that:

- Has limited success for tile constraints that have long range correlation length

Further, we:

- Provide the results of running a reference implementation on a variety of tile sets (Section 4)
- Explore the Tile Arc Consistent Correlation Length (TACCL), a heuristic for tile correlation length, that is used to inform the block size choice and solution strategies for a variety of tile sets (Section 4)

Here, a grid realization is a single tile assignment per grid cell that respects the tile constraints.

POMS works by initially setting the grid to an indeterminate state and progressively realizing sub blocks of the grid. Block boundary edges that fall within the larger grid body are *pinned* so that, should a block realization succeed, the block can be re-integrated back into the larger grid. Should block level realization fail, depending on the type of block realization failure, either the failed block region is set to an indeterminate state or the block region is restored to its previous state and all realized region boundaries within the grid are considered for *erosion* by probabilistically setting them to an indeterminate state.

https://zzyzek.github.io/ (. Z. Z.)
0009-0005-2175-1166 (. Z. Z.)

© 2024 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

POMS is a stochastic algorithm because of the reversion step. Undoing previous cell realizations, via block region reversion or erosion, are done in the hopes of removing a localized contradiction. Any expectation of progress for POMS primarily comes from choosing an appropriate block size. Conceptually, correlation length is the influence that a cell tile choice has over other cell tile options at distant locations. In this paper, we attempt to quantify an aspect of correlation length and use it to inform the block size choice.

If there is a finite length of correlation that one cell's tile choice has with another, any contradiction that might appear during the course of resolution are localized to a region. Reverting the region around the contradiction allows for another attempt at finding a realization without destroying the bulk of pre-existing realizations elsewhere in the grid. Under some conditions of configuration randomness, and for many tile constraints, resolved cells located far enough away from each other have little or no influence over one another. The correlation distance informs the block size choice as block sizes chosen large enough allow for the tile values in the middle cells of a block to be chosen independent of any tiles fixed on the boundary.

Expecting center tile choices to be independent of boundary tile values is, in general, unreasonable as the CBTG problem is known to be NP-Complete. Even under random configuration assumptions, the issue can be further complicated if the correlation length is not finite, or finite but large. Even though the general problem is likely intractable, or the finite correlation length assumption is violated, many tile constraints are under constrained and the ability of POMS to overcome local constraints provides enough capability to find realizations.

Unfortunately, for many tile sets, simple global constraints, tile distribution or sparse initial configuration restrictions are enough to confound POMS into failing to find a realization. Different choices for block scheduling policies, different block resolution algorithms and other parameter choices can help mitigate these shortcomings and will be briefly addressed (Section 4).

1.2. Definitions

We discuss some preliminary ideas before describing details of the Punch Out Model Synthesis (POMS) algorithm.

A *grid* is a collection of *cells* in the shape of a rectangular cuboid, of size $N_x \cdot N_y \cdot N_z$, ($N_x, N_y, N_z \in \mathbb{N}$).

Each cell in the grid is a variable whose domain is D *tiles*

Figure 1: Examples outputs of Punch Out Model Synthesis (POMS) run on different tile sets. From left to right, the *Pill Mortal* tile set, the *Forest Micro* tile set, the *Overhead Action RPG Overworld* tile set and the *Brutal Plum* tile set

($D \in \mathbb{N}$). Values in neighboring cells are subject to a set of provided *tile constraints*. Here, the set of tile constraints is pairwise, and only nearest neighbor in each of the major grid dimensions ($\pm X, \pm Y, \pm Z$).

A tile at a grid cell location is said to have *support from a direction* if there is at least one tile in the neighboring grid cell direction that respects the tile constraints. A tile, at a grid cell location, *has support* or *is supported* if a tile has support in each direction.

A cell is said to be *resolved* if there is only one tile present and the tile has support from its neighbors. Should every cell in the grid be resolved, the grid is said to be resolved. Should a cell hold no tiles, the grid is then said to be in a *contradictory* state.

The set of D tiles is called the *tile domain*. The set of available tile options at a cell is called the *cell's tile domain* and represents the available tiles that can be placed at a given cell location.

The problem of Constraint Based Tiling Generation (CBTG) is to find a single tile assignment to each grid cell location subject to the tile constraints. That is, resolve every cell in the grid.

It is sometimes desirable to pick out a sub region from a grid for special consideration. Here we identify a *block* as a sub region of cells from the grid. Our concern is with blocks that are rectangular cuboid in shape, of size $M_x, M_y, M_z \in \mathbb{N}$, and that can be smaller than the grid size ($1 \leq M_x \leq N_x, 1 \leq M_y \leq N_y, 1 \leq M_z \leq N_z$).

If every tile in every cell in a block is supported, the block of cells is said to be in an *Arc Consistent* (AC) state. That is, subject to the tile constraints, if every tile in every cell in a block region has at least one valid neighboring tile, the block region is said to be in an arc consistent state.

One method of attempting to put a block region into an arc consistent state is to remove tiles that have no support from the list of permissible tiles at a cell location. Each tile removed can have a cascading effect by potentially causing a tile in a neighboring cell to be unsupported. The repeated process of removing unsupported tiles throughout a block region until a contradiction is encountered or the block region is in an arc consistent state is called *constraint propagation*. Constraint propagation can be used as the basis for a Constraint Based Tiling Generation (CBTG) solver.

In this paper, we define two distinct classes of solvers that we call *block level* solvers and *grid level* solvers. We define a *block level* solver as an algorithm that keeps full state of the block it is trying to solve by propagating constraints and maintaining arc consistency throughout its run. A *grid level* solver need not keep full state of the grid and often will only keep minimal information about whether a grid cell is resolved or is in an indeterminate state. A block level solver typically needs more resources as, for example, it might maintain a memory intensive data structure associated with maintaining arc consistency.

POMS is a grid level solver with one of its input parameters designated to specify which underlying block level solver to use. In this paper we use *Breakout Model Synthesis* (BMS), a stochastic block level solver introduced in Hoetzlein's `just_math` project [1]. For completeness, pseudocode for *Breakout Model Synthesis* is given in Appendix A.

Note that since POMS is a grid level solver, the grid can be in an arc *inconsistent* state during the course of the algorithm. This poses no problem in and of itself as the block level solver will attempt to put the block in an arc consistent state while trying to make progress towards a fully resolved grid.

2. Related Work

To our knowledge, Merrell was the first to introduce the modern formulation of Constraint Based Tiling Generation (CBTG) [1] [3, 4]. Merrell introduced a Modify in Blocks Model Synthesis (MMS) algorithm that starts with a fully resolved grid and applies a one shot block level constraint solver on sub-blocks within the grid.

Merrell noticed that the block level algorithm undergoes a phase transition of solvability [2] , with a decreasing probability for successful block resolution as block size increases. Instead of attempting to resolve a large grid in one try, the MMS algorithm progressively resolves sub-blocks and reintegrates them back into the grid. If a block level resolution fails, the block is discarded without altering the grid and another block is chosen.

For many problems, MMS is ideal as it always keeps a full resolution of the grid throughout its run. Merrell introduced a sequential overlapping schedule for block choice and, for some suitable assumptions on block size and underlying distribution, the mixing time can be quick, requiring only a few passes through the grid.

Unfortunately, MMS has two major drawbacks, the second of which Merrell noticed and discussed in his thesis:

[1] The term *Constraint Based Tile Generators* was coined by Adam Newgas [2] and has been adopted in this paper, with slightly different wording, as the name for the specialization of the more general Constraint Satisfaction Problem.

[2] Note that the phase transition is only for *fallible* models. *Infallible* models will be discussed briefly later in this section.

- The requirement of an initially resolved state to start MMS might be difficult to achieve, either in a practical or theoretical sense.
- Features bigger than the chosen block size will be missed by MMS as there is no way to realize larger features through a single block level alteration.

Many of the tile sets and tile constraints that Merrell provides in [3, 4] have an "empty" tile that has itself as an admissible neighbor, creating a situation where the initial fully resolved grid can be easily created by populating each cell with an "empty" tile. All 2D tile sets and tile constraints presented in this paper do not have a valid resolution with a single replicated tile and require some amount of knowledge about the tile constraints to create a fully resolved configuration.

In general, for some tile constraints, finding a class of fully resolved initial configurations might be done through engineering effort. For example, it might be possible to find a small tileable block that is then able to be replicated through to the whole grid [3].

The inability to handle certain types of unbounded constraints, such as the implicit top and bottom equal river count constraint that appear in Woźniak's *Forest Micro* tile set (section 4), is the deeper issue with MMS. Choosing a small block size for MMS could miss novel features whereas too large of a block size either decreases the likelihood of block resolution or turns MMS into a block level solver.

Gumin introduced the Wave Function Collapse (WFC) project which improved the block level solver used in MMS and added facilities for automatic tile constraint deduction from exemplar scenes [5]. WFC uses a principle of maximum entropy heuristic [4] to choose which cells to resolve. Though extensions are possible, WFC as presented by Gumin is a one-shot block level solver, giving up should a contradiction be encountered.

Since MMS is a grid level solver, other block level solvers can be used, such as WFC, to resolve underlying blocks, with modifications added to allow for constraints, boundary conditions and other relevant features. Merrell provides a comparison between MMS and WFC in [6].

Breakout Model Synthesis (BMS) was introduced in Hoetzlein's `just_math` project [1]. Hoetzlein's `just_math` project also introduced the Tile Arc Consistent Correlation Length (TACCL) after noticing that knowledge of the tile correlation length could be used to create algorithms that took advantage of it. POMS takes the ideas of tile correlation and the TACCL to inform its stochastic backtracking strategies when applied to the larger grid without needing the resource requirements that BMS, as a block level solver, would require.

Table 1 provides a summary of the differences between WFC, BMS, MMS and POMS. Here, *contradiction resilience* means that the algorithm can recover should a contradiction be encountered, *block step consistent* means the algorithm is in an arc consistent state after every block resolution, *indeterminate initial state* means the algorithm doesn't require a fully resolved initial configuration and *ergodic* means that, in general, all solution states are possibly to reach. Note that an assertion of being *ergodic* in this context only means solutions are possible and does not mean *unbiased* as, depending on the tile constraints or configuration, solution biasing may occur. The features of Punch Out Model Synthesis (POMS) in Table 1 are highlighted for ease of comparison.

[3]This method was suggested through personal communication with P. Merrell.

[4]Using a *principle of maximum entropy* implies picking a cell to resolve of *minimum* entropy at each step.

	WFC	*BMS*	*MMS*	*POMS*
Solver Type	Block	Block	Grid	**Grid**
Contradiction Resilience	No	Yes	Yes	**Yes**
Block Step Consistent	n/a	n/a	Yes	***No***
Indeterminate Initial State	Yes	Yes	No	**Yes**
Ergodic	Yes	Yes	No	**Yes**

Table 1
WFC: Gumin's Wave Function Collapse
BMS : Breakout Model Synthesis
MMS: Merrell's Modify in Blocks Model Synthesis
POMS: Punch Out Model Synthesis (our algorithm)

In terms of algorithms to ensure arc consistency, Merrell's implementation of MMS [7] and Gumin's implementation of WFC [5] have used AC3 and AC4. AC3 is easy to understand and can be performant if tile count is low but quickly suffers as tile count increases [8]. AC4 is optimal, in general, but requires large amounts of auxiliary space [9]. The reference implementation of POMS uses AC4 exclusively as tile count is often large (1,000 or more).

CBTGs are a specialization of a more general Constraint Satisfaction Problem (CSP). Karth and Smith offer some history of CSPs, some common concepts and algorithms in [10, 11]. Of note is Karth and Smith's observation that shallow backtracking does little to help resolve conflicts.

Two areas of research activity for CBTGs are attempts to make infinite CBTG algorithms and attempts at giving more control over created output. Kleinberg provides an algorithm for infinite WFC by chaining blocks together [12]. While this can produce large scenes, the tile constraints are conditioned so that failure probability is low and Kleinberg admits that block resolution can fail without any recourse on how to continue.

Of note is Merrell's discussion of *infallible* models [4], where the tile constraints are conditioned so as to never be able to encounter a contradiction. For infallible models, infinite block chaining is always achievable as there is no possibility of a contradiction occurring. Though infallible models represent a case that is always solvable, it's unclear how possible or how difficult it is convert tile constraints from exemplar scenes to infallible models. In particular, all tile sets considered in this paper are fallible models.

Newgas provides `Tessera`, a software project that implements WFC along with options for a variety of constraints [13]. Nie et al. provide an extension to WFC to infinite grids but require the tile set to be *complete* or *sub-complete* [14] which may be difficult for tile sets in the wild. Cooper introduces `Sturgeon` that incorporates a mid level API to specify more explicit and longer range constraints as an addition to WFC [15]. Though Cooper's `Sturgeon` program and ideas look promising, the sizes involved are relatively small (40x40 and below) and it's unclear how well the constraints would work on various tile sets, initial conditions or how well the method would scale for larger level sizes.

Alaka and Bidarra attempt to offer more control over output level design by considering a user interface to group tiles and weight individual tile probabilities into user specified regions [16]. Langendam and Bidarra attempt to offer more